

Sprint 10 Structured Notes: Sets, Maps, WeakSets, and WeakMaps

0. What Sprint 10 Is About

Sprint 10 is about choosing the right JavaScript collection for the job.

Before this sprint, you mostly used arrays and objects:

```
const names = ["Ada", "Linus", "Grace"];

const user = {
  name: "Ada",
  score: 95
};
```

Arrays are useful when you need an ordered list.

Objects are useful when you need structured data with named properties.

Sprint 10 adds four more collection tools:

```
Set
Map
WeakSet
WeakMap
```

The main goal is:

Choose collection objects for unique values and key-value pairs.

By the end of Sprint 10, you should understand:

- what a `Set` is
- how `Set` removes duplicates
- `Set` methods: `add()`, `delete()`, `clear()`, `has()`, `entries()`, `forEach()`, `keys()`, `values()`, and `size`
- how to convert an array to a `Set`
- how to convert a `Set` back into an array
- what a `Map` is
- why `Map` can use keys of any type

- `Map` methods: `set()`, `get()`, `has()`, `delete()`, `clear()`, `entries()`, `forEach()`, and `size`
- how to loop through a `Map`
- how `Map` is different from a normal object
- what a `WeakSet` is
- what a `WeakMap` is
- what weak references are
- why `WeakSet` and `WeakMap` are not iterable
- when to use each collection type
- how to count item frequencies using `Map`
- common beginner mistakes with these collection types

1. Current Progress Before Sprint 10

Sprint 10 builds on the earlier sprints.

Earlier sprint	What you learned	How it helps with Sprint 10
Sprint 1	JavaScript can respond to actions on a web page.	Later, Sets and Maps can store data used by the page.
Sprint 2	Values, data types, <code>let</code> , <code>const</code> , primitives, and object references.	Sets and Maps store values. Weak collections depend on object references.
Sprint 3	Strings and text processing.	Sets can store strings and remove duplicate text values.
Sprint 4	Numbers, booleans, conditions, and comparisons.	You use conditions like <code>has()</code> checks when working with Sets and Maps.
Sprint 5	Functions, scope, callbacks, and closures.	Sets and Maps can be used inside functions and callbacks.
Sprint 6	Arrays, indexes, shallow copies, and spread syntax.	Sets often start from arrays, and spread syntax converts Sets back to arrays.
Sprint 7	Loops and iteration.	Sets and Maps can be looped through with <code>for...of</code> .
Sprint 8	Objects, key-value pairs, object references, and safe property access.	Maps are another way to store key-value pairs. WeakMaps use object keys.
Sprint 9	Higher-order array methods like <code>forEach()</code> , <code>map()</code> , <code>filter()</code> , and <code>reduce()</code> .	Sets and Maps also support <code>forEach()</code> , and Maps are useful for summary data.

Simple summary:

Before Sprint 10, you learned how to store and process values with arrays and objects. Sprint 10 teaches you special collection types for uniqueness, flexible keys, and object-based tracking.

Part A: Why New Collection Types Matter

2. The Problem with Arrays

Arrays are useful, but they allow duplicate values.

Example:

```
const names = ["Ada", "Linus", "Ada", "Grace"];  
  
console.log(names);
```

Output:

```
["Ada", "Linus", "Ada", "Grace"]
```

The name `"Ada"` appears twice.

Sometimes that is fine. For example, if you are recording every visit to a page, duplicates may be meaningful.

But sometimes duplicates are a problem. For example, if you are storing a list of unique students, duplicate names may be unwanted.

That is where `Set` helps.

3. The Problem with Objects as Lookup Tables

Objects store key-value pairs:

```
const scores = {  
  Ada: 95,  
  Linus: 88  
};
```

```
console.log(scores.Ada);
```

Output:

```
95
```

This works well when your keys are simple property names.

But objects are less convenient when your key is not a normal string-like property.

For example, what if the key is another object?

```
const user = { id: 1, name: "Ada" };
```

A normal object is awkward for this kind of key.

A `Map` is designed for this.

Simple idea:

Use `Set` when you care about unique values. Use `Map` when you care about key-value pairs with flexible keys.

Part B: Set

4. What Is a Set?

A `Set` is a collection that stores unique values.

Simple meaning:

A `Set` is like an array that refuses duplicates.

Example:

```
const names = new Set();  
  
names.add("Ada");  
names.add("Linus");
```

```
names.add("Ada");  
  
console.log(names);
```

Conceptual output:

```
Set { "Ada", "Linus" }
```

"Ada" was added twice, but the `Set` only kept one copy.

Important rule:

A `Set` ignores duplicate values.

5. Creating an Empty Set

Use `new Set()` to create an empty Set.

```
const skills = new Set();  
  
console.log(skills);
```

Conceptual output:

```
Set {}
```

Then you can add values with `add()`.

```
skills.add("HTML");  
skills.add("CSS");  
skills.add("JavaScript");  
  
console.log(skills);
```

Conceptual output:

```
Set { "HTML", "CSS", "JavaScript" }
```

6. Creating a Set from an Array

You can pass an array into `new Set()`.

```
const names = ["Ada", "Linus", "Ada", "Grace"];

const uniqueNames = new Set(names);

console.log(uniqueNames);
```

Conceptual output:

```
Set { "Ada", "Linus", "Grace" }
```

This is one of the most common beginner uses for `Set`.

Simple meaning:

```
new Set(names)
```

means:

Create a Set using the values from the `names` array.

Duplicates are removed automatically.

7. Converting a Set Back into an Array

A `Set` is not an array.

Sometimes you remove duplicates with a `Set`, but then you want an array again.

Use spread syntax:

```
const names = ["Ada", "Linus", "Ada", "Grace"];

const uniqueNames = [...new Set(names)];

console.log(uniqueNames);
```

Output:

```
["Ada", "Linus", "Grace"]
```

This part:

```
new Set(names)
```

creates a Set.

This part:

```
[...new Set(names)]
```

puts the Set values into a new array.

Simple meaning:

Make a Set to remove duplicates, then spread it into a new array.

8. Set Can Store Different Value Types

A `Set` can store values of any type.

Example:

```
const values = new Set();

values.add("Ada");
values.add(42);
values.add(true);
values.add(null);
values.add({ id: 1 });
values.add([1, 2, 3]);

console.log(values.size);
```

Output:

6

A `Set` can store:

- strings
- numbers
- booleans
- `null`
- `undefined`
- objects
- arrays
- functions
- symbols
- bigints

Beginner rule:

A `Set` can hold many types, but beginner code is usually clearer when one `Set` stores one kind of value.

Cleaner:

```
const uniqueNames = new Set(["Ada", "Linus", "Grace"]);
```

Less clear:

```
const mixedValues = new Set(["Ada", 42, true, null, { id: 1 }]);
```

Part C: Set Methods and Properties

9. `add()` — Add a Value

Use `add()` to add a value to a `Set`.

```
const skills = new Set();

skills.add("HTML");
skills.add("CSS");
```



```
console.log(skills);
```

Conceptual output:

```
Set { "HTML", "CSS" }
```

If you add the same value again, the Set does not create a duplicate.

```
skills.add("CSS");  
  
console.log(skills);
```

Conceptual output:

```
Set { "HTML", "CSS" }
```

Simple rule:

`add()` adds the value only if it is not already in the Set.

10. `has()` — Check If a Value Exists

Use `has()` to check whether a Set contains a value.

```
const skills = new Set(["HTML", "CSS", "JavaScript"]);  
  
console.log(skills.has("CSS"));  
console.log(skills.has("Python"));
```

Output:

```
true  
false
```

`has()` returns a boolean:

- `true` means the value exists
- `false` means the value does not exist

Common use with `if`:

```
if (skills.has("JavaScript")) {  
  console.log("JavaScript exists in the Set.");  
}
```

Output:

```
JavaScript exists in the Set.
```

11. `delete()` — Remove a Value

Use `delete()` to remove a value from a Set.

```
const skills = new Set(["HTML", "CSS", "JavaScript"]);  
  
skills.delete("CSS");  
  
console.log(skills.has("CSS"));  
console.log(skills.size);
```

Output:

```
false  
2
```

`delete()` also returns a boolean.

```
const skills = new Set(["HTML", "CSS"]);  
  
console.log(skills.delete("CSS"));  
console.log(skills.delete("Python"));
```

Output:

```
true  
false
```

Meaning:

- `true` means the value was found and removed
 - `false` means the value was not found
-

12. `clear()` — Remove Everything

Use `clear()` to remove all values from a Set.

```
const skills = new Set(["HTML", "CSS", "JavaScript"]);

skills.clear();

console.log(skills.size);
console.log(skills);
```

Output:

```
0
Set {}
```

Simple meaning:

`clear()` empties the whole Set.

13. `size` — Count Values

Use `size` to know how many unique values are in a Set.

```
const names = new Set(["Ada", "Linus", "Ada"]);

console.log(names.size);
```

Output:

```
2
```

Why `2`?

Because the Set contains only:

```
"Ada"  
"Linus"
```

Important beginner detail:

`size` is a property, not a method.

Correct:

```
console.log(names.size);
```

Wrong:

```
console.log(names.size());
```

Arrays use `.length`:

```
const arr = ["Ada", "Linus"];  
console.log(arr.length);
```

Sets use `.size`:

```
const set = new Set(["Ada", "Linus"]);  
console.log(set.size);
```

14. `values()` — Get Set Values

Use `values()` to get an iterator over the Set values.

```
const skills = new Set(["HTML", "CSS", "JavaScript"]);  
  
console.log(skills.values());
```

You usually use it with a loop:

```
for (const skill of skills.values()) {  
  console.log(skill);  
}
```

Output:

```
HTML  
CSS  
JavaScript
```

Because a Set only stores values, this is also equivalent:

```
for (const skill of skills) {  
  console.log(skill);  
}
```

15. `keys()` — Same as `values()` for Set

In a `Set`, `keys()` gives the same result as `values()`.

```
const skills = new Set(["HTML", "CSS", "JavaScript"]);  
  
for (const key of skills.keys()) {  
  console.log(key);  
}
```

Output:

```
HTML  
CSS  
JavaScript
```

This may look strange because Sets do not really have separate keys.

The reason `keys()` exists on Set is to keep Set similar to Map in the JavaScript API.

Beginner rule:

For Sets, use `values()` or just loop over the Set directly. You do not usually need `keys()`.

16. `entries()` — Get Set Entries

`entries()` returns pairs.

For a Set, each pair contains the same value twice.

```
const skills = new Set(["HTML", "CSS"]);

for (const entry of skills.entries()) {
  console.log(entry);
}
```

Output:

```
["HTML", "HTML"]
["CSS", "CSS"]
```

Why does it repeat the value?

Because a Set does not have separate keys and values.

So each entry is shaped like this:

```
[value, value]
```

Beginner rule:

`entries()` is more useful on `Map`. With `Set`, direct `for...of` is usually simpler.

17. `forEach()` — Run Code for Each Set Value

A Set has a `forEach()` method.

```
const skills = new Set(["HTML", "CSS", "JavaScript"]);

skills.forEach(skill => {
  console.log(skill);
});
```

Output:

```
HTML
CSS
JavaScript
```

Simple meaning:

For each value in the Set, run this callback function.

This connects to Sprint 9 because `forEach()` also uses a callback.

18. Set Method Summary

Method/property	What it does	Returns
<code>add(value)</code>	Adds a value	The Set itself
<code>has(value)</code>	Checks if a value exists	<code>true</code> or <code>false</code>
<code>delete(value)</code>	Removes a value	<code>true</code> or <code>false</code>
<code>clear()</code>	Removes all values	<code>undefined</code>
<code>size</code>	Counts unique values	A number
<code>values()</code>	Gets Set values	Iterator
<code>keys()</code>	Same as <code>values()</code> for Set	Iterator
<code>entries()</code>	Gets <code>[value, value]</code> pairs	Iterator
<code>forEach(callback)</code>	Runs a callback for each value	<code>undefined</code>

Part D: Looping Through a Set

19. Looping with `for...of`

A Set is iterable.

That means you can use `for...of`.

```
const skills = new Set(["HTML", "CSS", "JavaScript"]);

for (const skill of skills) {
  console.log(skill);
}
```

Output:

```
HTML
CSS
JavaScript
```

Use this when you want to read each value.

20. Looping with `forEach()`

You can also use `forEach()`.

```
const skills = new Set(["HTML", "CSS", "JavaScript"]);

skills.forEach(skill => {
  console.log(skill);
});
```

Output:

```
HTML
CSS
JavaScript
```

Comparison:

Loop style	Good use
<code>for...of</code>	Simple readable loop; can use <code>break</code> or <code>continue</code>
<code>forEach()</code>	Callback style; useful if matching other array-method code

Beginner rule:

Use `for...of` when you want the simplest loop.

Part E: Set Use Cases

21. Use Set to Remove Duplicates

Example:

```
const names = ["Ada", "Linus", "Ada", "Grace", "Linus"];

const uniqueNames = [...new Set(names)];

console.log(uniqueNames);
```

Output:

```
["Ada", "Linus", "Grace"]
```

This is the most important beginner Set pattern.

22. Use Set to Track Seen Values

A Set can track which values you already saw.

```
const seenNames = new Set();

const names = ["Ada", "Linus", "Ada", "Grace"];

for (const name of names) {
  if (seenNames.has(name)) {
    console.log(`${name} is a duplicate.`);
  } else {
    seenNames.add(name);
    console.log(`${name} is new.`);
  }
}
```

Output:

```
Ada is new.
Linus is new.
```

```
Ada is a duplicate.  
Grace is new.
```

This pattern is useful for duplicate detection.

23. Use Set for Fast Membership Checks

Membership means:

Is this value in the collection?

Example:

```
const allowedRoles = new Set(["admin", "editor", "viewer"]);  
  
const role = "editor";  
  
if (allowedRoles.has(role)) {  
  console.log("Role is allowed.");  
}
```

Output:

```
Role is allowed.
```

You could do this with an array using `includes()`, but a Set expresses the idea more directly:

This is a collection of allowed unique values.

Part F: Map

24. What Is a Map?

A `Map` stores key-value pairs.

Simple meaning:

A `Map` connects one value to another value.

Example:

```
const settings = new Map();

settings.set("theme", "dark");
settings.set("language", "JavaScript");

console.log(settings.get("theme"));
console.log(settings.get("language"));
```

Output:

```
dark
JavaScript
```

In this Map:

```
"theme" -> "dark"
"language" -> "JavaScript"
```

The first value is the key.

The second value is the stored value.

25. Creating an Empty Map

Use `new Map()`.

```
const scores = new Map();

console.log(scores);
```

Conceptual output:

```
Map {}
```

Then use `set()` to add key-value pairs.

```
scores.set("Ada", 95);
scores.set("Linus", 88);
```

```
console.log(scores);
```

Conceptual output:

```
Map { "Ada" => 95, "Linus" => 88 }
```

26. Creating a Map with Initial Entries

You can create a Map from an array of entry arrays.

Each entry is shaped like this:

```
[key, value]
```

Example:

```
const scores = new Map([
  ["Ada", 95],
  ["Linus", 88],
  ["Grace", 100]
]);

console.log(scores.get("Ada"));
```

Output:

```
95
```

The outer array contains entries.

```
[
  ["Ada", 95],
  ["Linus", 88],
  ["Grace", 100]
]
```

Each inner array is one key-value pair.

27. Map Keys Can Be Any Type

A major difference between `Map` and a normal object is this:

A `Map` can use keys of any type.

Map keys can be:

- strings
- numbers
- booleans
- objects
- arrays
- functions
- symbols

Example with string keys:

```
const settings = new Map();

settings.set("theme", "dark");
settings.set("language", "JavaScript");
```

Example with number keys:

```
const pages = new Map();

pages.set(1, "Introduction");
pages.set(2, "Main Content");

console.log(pages.get(1));
```

Output:

```
Introduction
```

Example with boolean keys:

```
const messages = new Map();

messages.set(true, "Enabled");
messages.set(false, "Disabled");
```

```
console.log(messages.get(false));
```

Output:

Disabled

Example with object keys:

```
const visits = new Map();  
  
const user = { id: 1, name: "Ada" };  
  
visits.set(user, 3);  
  
console.log(visits.get(user));
```

Output:

3

This is one of the most important Map features.

28. Object Keys in Map Depend on Reference Identity

When you use an object as a Map key, JavaScript checks the object reference.

Example:

```
const visits = new Map();  
  
const user = { id: 1 };  
  
visits.set(user, 3);  
  
console.log(visits.get(user));
```

Output:

3

This works because `get()` uses the exact same object reference.

But this does not work the same way:

```
const visits = new Map();  
visits.set({ id: 1 }, 3);  
console.log(visits.get({ id: 1 }));
```

Output:

undefined

Why?

Because these are two different object references:

```
{ id: 1 }  
{ id: 1 }
```

They look the same, but they are separate objects in memory.

Beginner rule:

When using an object as a Map key, you must use the same object reference to get the value back.

Part G: Map Methods and Properties

29. `set()` — Add or Update a Key-Value Pair

Use `set(key, value)` to add data to a Map.

```
const scores = new Map();  
scores.set("Ada", 95);
```

```
scores.set("Linus", 88);  
  
console.log(scores);
```

Conceptual output:

```
Map { "Ada" => 95, "Linus" => 88 }
```

If the key already exists, `set()` updates the value.

```
scores.set("Ada", 100);  
  
console.log(scores.get("Ada"));
```

Output:

```
100
```

Simple meaning:

`set()` adds a new pair or updates an existing key.

30. `get()` — Read a Value by Key

Use `get(key)` to read a value from a Map.

```
const scores = new Map();  
  
scores.set("Ada", 95);  
  
console.log(scores.get("Ada"));
```

Output:

```
95
```

If the key does not exist, `get()` returns `undefined`.


```
console.log(scores.get("Grace"));
```

Output:

```
undefined
```

Important:

```
scores.get("Grace")
```

asks:

What value is stored under the key "Grace" ?

31. `has()` — Check If a Key Exists

Use `has(key)` to check whether a Map has a key.

```
const scores = new Map();

scores.set("Ada", 95);

console.log(scores.has("Ada"));
console.log(scores.has("Grace"));
```

Output:

```
true
false
```

This is useful before reading or updating a value.

```
if (scores.has("Ada")) {
  console.log(scores.get("Ada"));
}
```

Output:

95

32. `delete()` — Remove a Key-Value Pair

Use `delete(key)` to remove one key-value pair.

```
const scores = new Map();

scores.set("Ada", 95);
scores.set("Linus", 88);

scores.delete("Linus");

console.log(scores.has("Linus"));
console.log(scores.size);
```

Output:

```
false
1
```

Like `Set.delete()`, `Map.delete()` returns a boolean.

```
console.log(scores.delete("Ada"));
console.log(scores.delete("Grace"));
```

Possible output:

```
true
false
```

Meaning:

- `true` means the key existed and was removed
- `false` means the key did not exist

33. `clear()` — Remove Everything

Use `clear()` to remove all key-value pairs from a Map.

```
const scores = new Map([
  ["Ada", 95],
  ["Linus", 88]
]);

scores.clear();

console.log(scores.size);
```

Output:

0

Simple meaning:

`clear()` empties the whole Map.

34. `size` — Count Key-Value Pairs

Use `size` to check how many pairs are in a Map.

```
const scores = new Map();

scores.set("Ada", 95);
scores.set("Linus", 88);
scores.set("Grace", 100);

console.log(scores.size);
```

Output:

3

Important:

`size` is a property, not a method.

Correct:

```
console.log(scores.size);
```

Wrong:

```
console.log(scores.size());
```

35. `entries()` — Get Key-Value Pairs

Use `entries()` to get Map entries.

Each entry is shaped like this:

```
[key, value]
```

Example:

```
const scores = new Map([
  ["Ada", 95],
  ["Linus", 88],
  ["Grace", 100]
]);

for (const entry of scores.entries()) {
  console.log(entry);
}
```

Output:

```
["Ada", 95]
["Linus", 88]
["Grace", 100]
```

You can also loop directly over the Map:

```
for (const entry of scores) {  
  console.log(entry);  
}
```

That gives the same kind of entries.

36. Destructuring Map Entries

Because each Map entry is an array like this:

```
["Ada", 95]
```

You can destructure it:

```
const [name, score] = ["Ada", 95];  
  
console.log(name);  
console.log(score);
```

Output:

```
Ada  
95
```

This is useful in Map loops:

```
const scores = new Map([  
  ["Ada", 95],  
  ["Linus", 88],  
  ["Grace", 100]  
]);  
  
for (const [name, score] of scores) {  
  console.log(`${name}: ${score}`);  
}
```

Output:

```
Ada: 95  
Linus: 88  
Grace: 100
```

Simple meaning:

```
for (const [name, score] of scores)
```

means:

For each key-value pair in the Map, call the key `name` and the value `score`.

37. `forEach()` — Run Code for Each Map Pair

A Map also has `forEach()`.

```
const scores = new Map([  
  ["Ada", 95],  
  ["Linus", 88]  
]);  
  
scores.forEach((score, name) => {  
  console.log(`${name}: ${score}`);  
});
```

Output:

```
Ada: 95  
Linus: 88
```

Important detail:

For Map `forEach()`, the callback receives the value first and the key second.

```
(value, key) => {  
  // code here  
}
```

Example:

```
scores.forEach((value, key) => {  
  console.log(key, value);  
});
```

Beginner warning:

In a `for...of` loop, you usually write `[key, value]`. In `Map.forEach()`, the callback parameters are usually `(value, key)`.

38. Map Method Summary

Method/property	What it does	Returns
<code>set(key, value)</code>	Adds or updates a pair	The Map itself
<code>get(key)</code>	Reads a value by key	The value or <code>undefined</code>
<code>has(key)</code>	Checks if a key exists	<code>true</code> or <code>false</code>
<code>delete(key)</code>	Removes a pair	<code>true</code> or <code>false</code>
<code>clear()</code>	Removes all pairs	<code>undefined</code>
<code>size</code>	Counts pairs	A number
<code>entries()</code>	Gets <code>[key, value]</code> pairs	Iterator
<code>forEach(callback)</code>	Runs a callback for each pair	<code>undefined</code>

Part H: Map vs Object

39. Objects Are Good for Structured Data

Objects are excellent when you are describing one thing.

Example:

```
const user = {  
  name: "Ada",  
  age: 25,  
  active: true  
};
```

This object represents one user.

Use objects for structured records like:

- users
 - products
 - tasks
 - books
 - settings objects
 - form data
-

40. Maps Are Good for Lookup Tables

A Map is good when you are connecting keys to values.

Example:

```
const scores = new Map();

scores.set("Ada", 95);
scores.set("Linus", 88);
scores.set("Grace", 100);
```

This Map works like a lookup table:

```
"Ada" -> 95
"Linus" -> 88
"Grace" -> 100
```

Use Maps for:

- counts
 - lookup tables
 - cache-like storage
 - object keys
 - situations where keys are not simple property names
-

41. Map vs Object Comparison

Question	Object	Map
Main purpose	Structured data	Key-value collection
Key types	Usually strings or symbols	Any type
Object keys?	Awkward	Natural
Size count	Use <code>Object.keys(obj).length</code>	Use <code>.size</code>
Iteration	Needs <code>Object.keys()</code> , <code>Object.values()</code> , or <code>Object.entries()</code>	Directly iterable
Read syntax	<code>obj.key</code> or <code>obj["key"]</code>	<code>map.get(key)</code>
Write syntax	<code>obj.key = value</code>	<code>map.set(key, value)</code>

Beginner rule:

Use an object to describe one thing. Use a Map when you need a flexible key-value collection.

42. Wrong: Reading a Map Like an Object

This is a common beginner mistake.

Wrong:

```
const settings = new Map();

settings.set("theme", "dark");

console.log(settings.theme);
```

Output:

```
undefined
```

Why?

Because a Map does not store normal properties in the same way an object does.

Correct:

```
console.log(settings.get("theme"));
```

Output:

```
dark
```

Beginner rule:

Use `.get(key)` to read from a Map.

Part I: Weak References

43. Strong References

Most normal variables and collections keep strong references to objects.

Example:

```
const user = { name: "Ada" };  
const users = new Set();  
  
users.add(user);
```

The Set strongly references the `user` object.

Simple meaning:

As long as the Set keeps that object, JavaScript must keep the object in memory.

This is normal and usually fine.

44. Weak References

A weak reference does not prevent an object from being cleaned up by JavaScript.

Simple meaning:

If nothing else in the program is using an object, JavaScript is allowed to remove it from memory, even if it is inside a weak collection.

Weak references are used by:

WeakSet
WeakMap

This is mainly about memory management.

Beginner rule:

You do not usually need weak collections in simple beginner projects. Learn what they are, but use `Set` and `Map` most of the time.

45. Garbage Collection

Garbage collection is JavaScript's automatic memory cleanup process.

Simple meaning:

JavaScript removes objects from memory when they are no longer reachable or needed.

You do not manually delete memory in normal JavaScript.

Weak collections are connected to garbage collection because they do not strongly hold objects in memory.

Part J: WeakSet

46. What Is a WeakSet?

A `WeakSet` is similar to a `Set`, but it has stricter rules.

Simple meaning:

A `WeakSet` is used to track objects weakly.

Example:

```
const visitedUsers = new WeakSet();

const user = { id: 1, name: "Ada" };

visitedUsers.add(user);

console.log(visitedUsers.has(user));
```

Output:

```
true
```

The WeakSet tracks that this user object has been visited.

47. WeakSet Values Must Be Objects

A `WeakSet` cannot store primitive strings or numbers.

This fails:

```
const weak = new WeakSet();

weak.add("Ada");
```

Why?

Because `"Ada"` is a primitive string, not an object.

This also fails:

```
const weak = new WeakSet();

weak.add(42);
```

Why?

Because `42` is a primitive number.

This works:

```
const weak = new WeakSet();

const user = { name: "Ada" };

weak.add(user);

console.log(weak.has(user));
```

Output:

```
true
```

Advanced note:

WeakSets can also store symbols in modern JavaScript when they are garbage-collectable symbols. As a beginner, think of WeakSet as object-focused.

48. WeakSet Methods

A WeakSet has fewer methods than a Set.

Common WeakSet methods:

Method	Meaning
<code>add(object)</code>	Add an object
<code>has(object)</code>	Check if an object exists
<code>delete(object)</code>	Remove an object

Example:

```
const activeUsers = new WeakSet();

const user = { id: 1 };

activeUsers.add(user);

console.log(activeUsers.has(user));

activeUsers.delete(user);
```

```
console.log(activeUsers.has(user));
```

Output:

```
true  
false
```

49. WeakSet Is Not Iterable

You cannot loop through a WeakSet.

Wrong:

```
const weak = new WeakSet();  
  
const user = { id: 1 };  
weak.add(user);  
  
for (const item of weak) {  
  console.log(item);  
}
```

This does not work.

Why?

Because WeakSet is not iterable.

It also does not have `size`.

Wrong idea:

```
console.log(weak.size);
```

This does not give you a useful count.

Why is WeakSet not iterable?

Because JavaScript may clean up weakly held objects at any time when nothing else references them.

If JavaScript allowed you to list everything in a WeakSet, the result could be unreliable.

Beginner rule:

With WeakSet, you can ask whether a specific object is in it, but you cannot list all objects inside it.

50. WeakSet Use Case: Object Tracking

A WeakSet can track whether an object has already been processed.

```
const processedUsers = new WeakSet();

function processUser(user) {
  if (processedUsers.has(user)) {
    return "Already processed";
  }

  processedUsers.add(user);
  return "Processing user";
}

const user = { id: 1, name: "Ada" };

console.log(processUser(user));
console.log(processUser(user));
```

Output:

```
Processing user
Already processed
```

This is useful when tracking object identity without preventing memory cleanup.

Beginner note:

You will not use WeakSet often at first. Understand it as a special tool for object tracking.

Part K: WeakMap

51. What Is a WeakMap?

A `WeakMap` is similar to a `Map`, but its keys must be objects.

Simple meaning:

A `WeakMap` stores values connected to object keys, using weak references.

Example:

```
const privateData = new WeakMap();

const user = { id: 1 };

privateData.set(user, {
  passwordHint: "favorite color"
});

console.log(privateData.get(user));
```

Output:

```
{ passwordHint: "favorite color" }
```

Here:

```
user -> { passwordHint: "favorite color" }
```

The key is the `user` object.

The value is another object with extra data.

52. WeakMap Keys Must Be Objects

This works:

```
const weakMap = new WeakMap();
```



```
const user = { id: 1 };  
  
weakMap.set(user, "active");  
  
console.log(weakMap.get(user));
```

Output:

```
active
```

This fails:

```
const weakMap = new WeakMap();  
  
weakMap.set("user", "active");
```

Why?

Because `"user"` is a primitive string, not an object.

This also fails:

```
weakMap.set(1, "active");
```

Why?

Because `1` is a primitive number, not an object.

Beginner rule:

WeakMap keys must be objects.

53. WeakMap Values Can Be Any Type

In a WeakMap, the key must be an object.

But the value can be many types.

Example:

```
const weakMap = new WeakMap();

const user = { id: 1 };

weakMap.set(user, "active");
weakMap.set(user, 100);
weakMap.set(user, { role: "admin" });
```

The key is still the `user` object.

The value can be a string, number, object, array, function, or other value.

54. WeakMap Methods

A WeakMap has fewer methods than a Map.

Common WeakMap methods:

Method	Meaning
<code>set(objectKey, value)</code>	Add or update a value for an object key
<code>get(objectKey)</code>	Read a value by object key
<code>has(objectKey)</code>	Check if an object key exists
<code>delete(objectKey)</code>	Remove an object key and its value

Example:

```
const data = new WeakMap();

const user = { id: 1 };

data.set(user, "extra information");

console.log(data.has(user));
console.log(data.get(user));

data.delete(user);

console.log(data.has(user));
```

Output:

```
true  
extra information  
false
```

55. WeakMap Is Not Iterable

You cannot loop through a WeakMap.

Wrong:

```
const weakMap = new WeakMap();  
  
const user = { id: 1 };  
weakMap.set(user, "active");  
  
for (const entry of weakMap) {  
  console.log(entry);  
}
```

This does not work.

WeakMap also does not have `size`.

Wrong idea:

```
console.log(weakMap.size);
```

Why?

Because WeakMap keys are weakly held. JavaScript may clean them up if nothing else references them.

Beginner rule:

With WeakMap, you can work with a specific object key, but you cannot list all entries.

56. WeakMap Use Case: Private-Like Object Data

WeakMap is useful when you want to store extra data connected to an object without putting that data directly on the object.

Example:

```
const privateScores = new WeakMap();

function createStudent(name, score) {
  const student = { name };

  privateScores.set(student, score);

  return student;
}

const ada = createStudent("Ada", 95);

console.log(ada);
console.log(privateScores.get(ada));
```

Output:

```
{ name: "Ada" }
95
```

The object only visibly contains:

```
{ name: "Ada" }
```

The score is stored separately in the WeakMap.

This is why WeakMap is sometimes described as useful for private-like data.

Beginner warning:

Do not overuse WeakMap. For most beginner projects, a normal object property is simpler.

Part L: Set vs Map vs WeakSet vs WeakMap

57. Main Comparison

Tool	Stores	Allows primitive values?	Uses keys?	Iterable?	Has size?	Main beginner use
<code>Set</code>	Unique values	Yes	No separate keys	Yes	Yes	Remove duplicates
<code>Map</code>	Key-value pairs	Yes	Yes, keys can be any type	Yes	Yes	Lookup data with flexible keys
<code>WeakSet</code>	Weakly held objects	No primitive strings/ numbers	No separate keys	No	No	Track objects privately
<code>WeakMap</code>	Values connected to weakly held object keys	Keys cannot be primitive strings/ numbers	Object keys only	No	No	Attach private-like data to objects

58. Decision Rule

Use this rule when choosing a collection:

Need unique values?	Use <code>Set</code> .
Need to remove duplicates?	Use <code>Set</code> .
Need key-value pairs?	Use <code>Map</code> .
Need object keys?	Use <code>Map</code> .
Need memory-friendly tracking?	Maybe <code>WeakSet</code> .
Need private-like object data?	Maybe <code>WeakMap</code> .

Most beginner projects should mainly use:

`Set`
`Map`

Weak collections are special-purpose tools.

59. Quick Examples

Need unique names:

```
const uniqueNames = new Set(names);
```

Need a list without duplicates:

```
const cleanNames = [...new Set(names)];
```

Need to count how many times each item appears:

```
const counts = new Map();
```

Need to use an object as a key:

```
const visits = new Map();  
visits.set(userObject, 3);
```

Need to track whether an object was processed:

```
const processed = new WeakSet();  
processed.add(userObject);
```

Need private-like data attached to an object:

```
const privateData = new WeakMap();  
privateData.set(userObject, secretData);
```

Part M: Mini-Build 1 — Remove Duplicate Names with Set

60. Problem

You have an array with duplicate names:

```
const names = ["Ada", "Linus", "Ada", "Grace", "Linus"];
```

You want this:

```
["Ada", "Linus", "Grace"]
```

61. Solution

```
const names = ["Ada", "Linus", "Ada", "Grace", "Linus"];

const uniqueNames = [...new Set(names)];

console.log(uniqueNames);
```

Output:

```
["Ada", "Linus", "Grace"]
```

62. Step-by-Step Explanation

Start:

```
["Ada", "Linus", "Ada", "Grace", "Linus"]
```

Create Set:

```
new Set(names)
```

The Set keeps only:

```
"Ada"
"Linus"
"Grace"
```

Convert back to array:

```
[...new Set(names)]
```

Final result:

```
["Ada", "Linus", "Grace"]
```

Part N: Mini-Build 2 — Store Object Keys with Map

63. Problem

You have a user object:

```
const user = { id: 1 };
```

You want to store the number of visits for that exact object.

64. Solution

```
const visits = new Map();  
  
const user = { id: 1 };  
  
visits.set(user, 3);  
  
console.log(visits.get(user));
```

Output:

```
3
```

65. Step-by-Step Explanation

Create a Map:


```
const visits = new Map();
```

Create an object:

```
const user = { id: 1 };
```

Use the object as the key:

```
visits.set(user, 3);
```

Read the value using the same object:

```
visits.get(user)
```

Output:

```
3
```

Important:

The same object reference must be used to read the value back.

Part O: Mini-Build 3 — Count Item Frequencies with Map

66. Problem

You have repeated items:

```
const items = ["apple", "banana", "apple", "orange", "banana", "apple"];
```

You want to count how many times each item appears.

Wanted result:

```
apple: 3  
banana: 2  
orange: 1
```

67. Solution

```
const items = ["apple", "banana", "apple", "orange", "banana", "apple"];  
  
const counts = new Map();  
  
for (const item of items) {  
  if (counts.has(item)) {  
    counts.set(item, counts.get(item) + 1);  
  } else {  
    counts.set(item, 1);  
  }  
}  
  
for (const [item, count] of counts) {  
  console.log(`${item}: ${count}`);  
}
```

Output:

```
apple: 3  
banana: 2  
orange: 1
```

68. Step-by-Step Explanation

Start with the array:

```
["apple", "banana", "apple", "orange", "banana", "apple"]
```

Create an empty Map:

```
const counts = new Map();
```

Loop through each item:

```
for (const item of items) {  
  // code here  
}
```

For each item, ask:

```
counts.has(item)
```

Simple meaning:

Have I seen this item before?

If yes:

```
counts.set(item, counts.get(item) + 1);
```

Simple meaning:

Read the old count, add 1, and save it again.

If no:

```
counts.set(item, 1);
```

Simple meaning:

This is the first time seeing this item, so start its count at 1.

69. Dry Run

Initial Map:

```
Map {}
```

Process "apple":

```
apple -> 1
```

Process "banana":

```
apple -> 1  
banana -> 1
```

Process second "apple":

```
apple -> 2  
banana -> 1
```

Process "orange":

```
apple -> 2  
banana -> 1  
orange -> 1
```

Process second "banana":

```
apple -> 2  
banana -> 2  
orange -> 1
```

Process third "apple":

```
apple -> 3  
banana -> 2  
orange -> 1
```

Final result:

```
apple: 3  
banana: 2  
orange: 1
```

Part P: Extra Challenge — Convert a Map to an Array and Sort It

70. Problem

You have a Map of counts:

```
const counts = new Map([
  ["apple", 3],
  ["banana", 2],
  ["orange", 1]
]);
```

You want to convert it into an array of entries and sort it by count.

71. Convert Map to Array

Use spread syntax:

```
const entries = [...counts];

console.log(entries);
```

Output:

```
[
  ["apple", 3],
  ["banana", 2],
  ["orange", 1]
]
```

This works because Maps are iterable.

72. Sort by Count Ascending

Ascending means smallest to largest.

```
const counts = new Map([
  ["apple", 3],
  ["banana", 2],
  ["orange", 1]
]);

const sortedEntries = [...counts].sort((a, b) => a[1] - b[1]);

console.log(sortedEntries);
```

Output:

```
[
  ["orange", 1],
  ["banana", 2],
  ["apple", 3]
]
```

Why `a[1]` and `b[1]`?

Each entry looks like this:

```
["apple", 3]
```

Index `0` is the key:

```
"apple"
```

Index `1` is the count:

```
3
```

So:

```
a[1] - b[1]
```

sorts by the count.

73. Sort by Count Descending

Descending means largest to smallest.

```
const sortedEntries = [...counts].sort((a, b) => b[1] - a[1]);  
  
console.log(sortedEntries);
```

Output:

```
[  
  ["apple", 3],  
  ["banana", 2],  
  ["orange", 1]  
]
```

Part Q: Common Beginner Mistakes

74. Mistake 1 — Expecting Set to Keep Duplicates

Wrong expectation:

```
const set = new Set();  
  
set.add("Ada");  
set.add("Ada");  
  
console.log(set.size);
```

Output:

```
1
```

Why?

Because a Set only keeps unique values.

Correct understanding:

A duplicate value is ignored.

75. Mistake 2 — Using `get()` on a Set

Wrong:

```
const names = new Set(["Ada", "Grace"]);  
  
console.log(names.get("Ada"));
```

Problem:

`Set` does not have `get()`.

A Set does not store separate keys and values.

Correct:

```
console.log(names.has("Ada"));
```

Output:

```
true
```

Use `has()` to check whether a value exists.

76. Mistake 3 — Using Object-Style Access with Map

Wrong:

```
const map = new Map();  
  
map.set("theme", "dark");  
  
console.log(map.theme);
```

Output:


```
undefined
```

Problem:

A Map is not read with dot notation.

Correct:

```
console.log(map.get("theme"));
```

Output:

```
dark
```

Beginner rule:

Use `.get(key)` to read from a Map.

77. Mistake 4 — Forgetting That Map `set()` Updates Existing Keys

Example:

```
const scores = new Map();

scores.set("Ada", 95);
scores.set("Ada", 100);

console.log(scores.size);
console.log(scores.get("Ada"));
```

Output:

```
1
100
```

Why?

Because the second `set()` updated the existing key.

It did not create a duplicate key.

78. Mistake 5 — Using a New Object Instead of the Same Object Key

Wrong:

```
const visits = new Map();

visits.set({ id: 1 }, 3);

console.log(visits.get({ id: 1 }));
```

Output:

```
undefined
```

Problem:

The two object literals are different object references.

Correct:

```
const visits = new Map();

const user = { id: 1 };

visits.set(user, 3);

console.log(visits.get(user));
```

Output:

```
3
```

Beginner rule:

For object keys, use the same object reference.

79. Mistake 6 — Adding a Primitive Value to WeakSet

Wrong:

```
const weak = new WeakSet();  
  
weak.add("Ada");
```

Problem:

"Ada" is a primitive string.

WeakSet is for object-based tracking.

Correct:

```
const weak = new WeakSet();  
  
const user = { name: "Ada" };  
  
weak.add(user);  
  
console.log(weak.has(user));
```

Output:

```
true
```

80. Mistake 7 — Using a Primitive Key in WeakMap

Wrong:

```
const weakMap = new WeakMap();  
  
weakMap.set("user", "active");
```

Problem:

WeakMap keys must be objects.

Correct:

```
const weakMap = new WeakMap();

const user = { id: 1 };

weakMap.set(user, "active");

console.log(weakMap.get(user));
```

Output:

```
active
```

81. Mistake 8 — Trying to Loop Through WeakSet or WeakMap

Wrong:

```
const weakMap = new WeakMap();

for (const item of weakMap) {
  console.log(item);
}
```

This does not work.

Why?

WeakMap is not iterable.

WeakSet is also not iterable.

Correct understanding:

You cannot list everything inside a WeakMap or WeakSet.

You can only check a specific object:

```
weakMap.has(user);  
weakSet.has(user);
```

82. Mistake 9 — Expecting WeakMap or WeakSet to Have `size`

Wrong idea:

```
console.log(weakMap.size);  
console.log(weakSet.size);
```

Problem:

WeakMap and WeakSet do not expose `size`.

Why?

Because their contents are weakly held and may be garbage-collected.

Correct understanding:

Weak collections do not let you count or list everything inside them.

Part R: Practice Tasks

83. Practice Task 1 — Remove Duplicate Names

Data:

```
const names = ["Ada", "Linus", "Ada", "Grace"];
```

Task:

Create a new array with duplicates removed.

Solution:

```
const names = ["Ada", "Linus", "Ada", "Grace"];
```

```
const uniqueNames = [...new Set(names)];  
  
console.log(uniqueNames);
```

Expected output:

```
["Ada", "Linus", "Grace"]
```

84. Practice Task 2 — Check If a Skill Exists

Data:

```
const skills = new Set(["HTML", "CSS", "JavaScript"]);
```

Task:

Check whether `"JavaScript"` exists.

Solution:

```
const skills = new Set(["HTML", "CSS", "JavaScript"]);  
  
console.log(skills.has("JavaScript"));
```

Expected output:

```
true
```

85. Practice Task 3 — Store Scores with Map

Task:

Create a Map that stores student names and scores.

Solution:

```
const scores = new Map();

scores.set("Ada", 95);
scores.set("Linus", 88);
scores.set("Grace", 100);

console.log(scores.get("Ada"));
console.log(scores.size);
```

Expected output:

```
95
3
```

86. Practice Task 4 — Loop Through a Map

Data:

```
const scores = new Map([
  ["Ada", 95],
  ["Linus", 88],
  ["Grace", 100]
]);
```

Task:

Print each name and score.

Solution:

```
const scores = new Map([
  ["Ada", 95],
  ["Linus", 88],
  ["Grace", 100]
]);

for (const [name, score] of scores) {
  console.log(`${name}: ${score}`);
}
```

Expected output:

```
Ada: 95  
Linus: 88  
Grace: 100
```

87. Practice Task 5 — Count Frequencies with Map

Data:

```
const items = ["apple", "banana", "apple", "orange", "banana", "apple"];
```

Task:

Count how many times each item appears.

Solution:

```
const items = ["apple", "banana", "apple", "orange", "banana", "apple"];  
  
const counts = new Map();  
  
for (const item of items) {  
  if (counts.has(item)) {  
    counts.set(item, counts.get(item) + 1);  
  } else {  
    counts.set(item, 1);  
  }  
}  
  
console.log(counts);
```

Conceptual output:

```
Map { "apple" => 3, "banana" => 2, "orange" => 1 }
```

88. Practice Task 6 — Debug WeakSet Error

Broken code:


```
const weak = new WeakSet();

weak.add("Ada");
```

Problem:

"Ada" is a primitive string.

WeakSet needs object-based values.

Fixed code:

```
const weak = new WeakSet();

const user = { name: "Ada" };

weak.add(user);

console.log(weak.has(user));
```

Expected output:

```
true
```

Part S: Sprint 10 Checkpoint Questions

89. Questions

You are ready to move on when you can answer these:

1. What is a `Set`?
2. What happens when you add a duplicate value to a `Set`?
3. How do you create an empty `Set`?
4. How do you create a `Set` from an array?
5. How do you convert a `Set` back into an array?
6. What does `set.add(value)` do?
7. What does `set.has(value)` return?
8. What does `set.delete(value)` return?
9. What does `set.clear()` do?
10. Why is `set.size` not written as `set.size()`?

11. What does `Set.prototype.entries()` return?
 12. Why are `keys()` and `values()` basically the same for a Set?
 13. What is a `Map`?
 14. What does `map.set(key, value)` do?
 15. What does `map.get(key)` return if the key does not exist?
 16. Why can a Map use an object as a key?
 17. Why must you use the same object reference when reading from a Map with an object key?
 18. What is the difference between a normal object and a Map?
 19. How do you loop through a Map with `for...of`?
 20. What does `map.entries()` return?
 21. What is a weak reference?
 22. What is garbage collection?
 23. What is a WeakSet?
 24. Why does `weakSet.add("Ada")` fail?
 25. Why can you not loop through a WeakSet?
 26. What is a WeakMap?
 27. Why must WeakMap keys be objects?
 28. Why does WeakMap not have `size`?
 29. When should you use Set?
 30. When should you use Map?
 31. When might you use WeakSet?
 32. When might you use WeakMap?
 33. How can a Map count item frequencies?
 34. How do you convert a Map into an array of entries?
 35. How do you sort Map entries by value?
-

90. Short Answers

1. A `Set` stores unique values.
2. Duplicate values are ignored.
3. Use `new Set()`.
4. Use `new Set(array)`.
5. Use `[...set]`.
6. `add()` adds a value to a Set.
7. `has()` returns `true` or `false`.
8. `delete()` returns `true` if something was removed, otherwise `false`.
9. `clear()` removes everything.
10. Because `size` is a property, not a method.
11. Set `entries()` returns `[value, value]` pairs.
12. Because Set has values, not separate keys.
13. A `Map` stores key-value pairs.
14. `set()` adds or updates a pair.
15. `get()` returns `undefined` when the key does not exist.
16. A Map can use keys of any type, including objects.
17. Object keys are matched by reference identity.

18. Objects describe structured data; Maps are flexible key-value collections.
 19. Use `for (const [key, value] of map)`.
 20. Map `entries()` returns `[key, value]` pairs.
 21. A weak reference does not stop garbage collection.
 22. Garbage collection is JavaScript's automatic memory cleanup.
 23. A WeakSet tracks objects weakly.
 24. Because `"Ada"` is a primitive string, not an object.
 25. WeakSet is not iterable.
 26. A WeakMap stores values connected to object keys weakly.
 27. WeakMap keys must be objects because weak references work with garbage-collectable keys.
 28. Its contents can change due to garbage collection, so it does not expose a reliable count.
 29. Use Set for unique values and duplicate removal.
 30. Use Map for key-value pairs and flexible keys.
 31. Use WeakSet for memory-friendly object tracking.
 32. Use WeakMap for private-like data connected to object keys.
 33. Use each item as a key and its count as the value.
 34. Use `[...map]` or `[...map.entries()]`.
 35. Use `.sort((a, b) => a[1] - b[1])` for ascending values.
-

Part T: Final Sprint 10 Mental Model

91. The Main Idea

Sprint 10 gives you four collection tools:

Set
Map
WeakSet
WeakMap

The most important beginner pair is:

Set
Map

Use `Set` when the main question is:

Do I have each value only once?

Use `Map` when the main question is:

What value belongs to this key?

Use `WeakSet` when the main question is:

Have I seen this object before, without forcing it to stay in memory?

Use `WeakMap` when the main question is:

What private-like data belongs to this object, without forcing the object to stay in memory?

92. Final Beginner Rule

<code>Set</code>	= unique values
<code>Map</code>	= key-value pairs
<code>WeakSet</code>	= weak object tracking
<code>WeakMap</code>	= weak object-key data

Most of the time:

Use `Set` for uniqueness.
Use `Map` for lookup.
Know `WeakSet` and `WeakMap` exist, but do not overuse them yet.

93. Minimum Code Patterns to Memorize

Remove duplicates

```
const uniqueValues = [...new Set(values)];
```

Create and use a Set

```
const set = new Set();  
  
set.add("Ada");
```

```
console.log(set.has("Ada"));
console.log(set.size);
```

Create and use a Map

```
const map = new Map();

map.set("theme", "dark");
console.log(map.get("theme"));
console.log(map.size);
```

Loop through a Map

```
for (const [key, value] of map) {
  console.log(key, value);
}
```

Count frequencies

```
const counts = new Map();

for (const item of items) {
  if (counts.has(item)) {
    counts.set(item, counts.get(item) + 1);
  } else {
    counts.set(item, 1);
  }
}
```

Use WeakSet with an object

```
const weakSet = new WeakSet();
const user = { id: 1 };

weakSet.add(user);
console.log(weakSet.has(user));
```

Use WeakMap with an object key

```
const weakMap = new WeakMap();
const user = { id: 1 };
```

```
weakMap.set(user, "private data");  
console.log(weakMap.get(user));
```

94. Sprint 10 Completion Standard

You have completed Sprint 10 when you can do these without copying blindly:

- explain why a Set removes duplicates
- use `add()`, `has()`, `delete()`, `clear()`, and `size` on a Set
- convert an array to a Set
- convert a Set back to an array
- explain why Set `entries()` gives `[value, value]`
- use `set()`, `get()`, `has()`, `delete()`, `clear()`, and `size` on a Map
- loop through a Map with destructuring
- explain why Maps can use object keys
- explain why object keys depend on reference identity
- explain why WeakSet cannot store primitive strings or numbers
- explain why WeakMap keys must be objects
- explain why WeakSet and WeakMap are not iterable
- choose Set, Map, WeakSet, or WeakMap for a simple problem
- build a frequency counter using Map

Final checkpoint:

```
Set: unique values.  
Map: flexible key-value pairs.  
WeakSet: weak object tracking.  
WeakMap: weak object-key data.
```